



CSE 4513

Lec – 6

Software Design (Basics)



If you give me a program that **works** perfectly but is impossible to change, then it won't work when the requirements change, and I won't be able to make it work.
Therefore the program will become useless.

If you give me a program that **does not work** but is easy to change, then I can make it work, and keep it working as requirements change. Therefore the program will remain continually useful.

WHAT IS SOFTWARE DESIGN



is a mechanism to **transform user requirements** into some suitable form, which **helps the programmer in software coding and implementation**.

- ✓ It deals with **representing the client's requirement**, as described in SRS (Software Requirement Specification) document, into a form, i.e., easily implementable using programming language.
- ✓ moves the concentration from the **problem domain to the solution domain**.
- ✓ In software design, we consider the system to be a **set of components or modules** with **clearly defined behaviors & boundaries**.

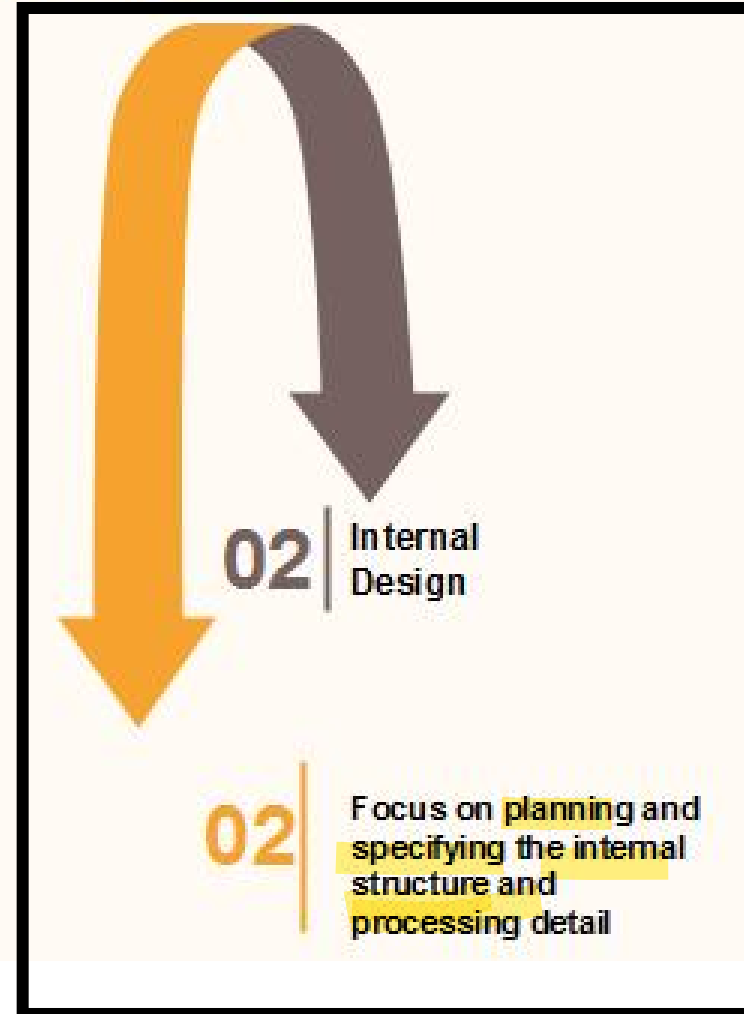
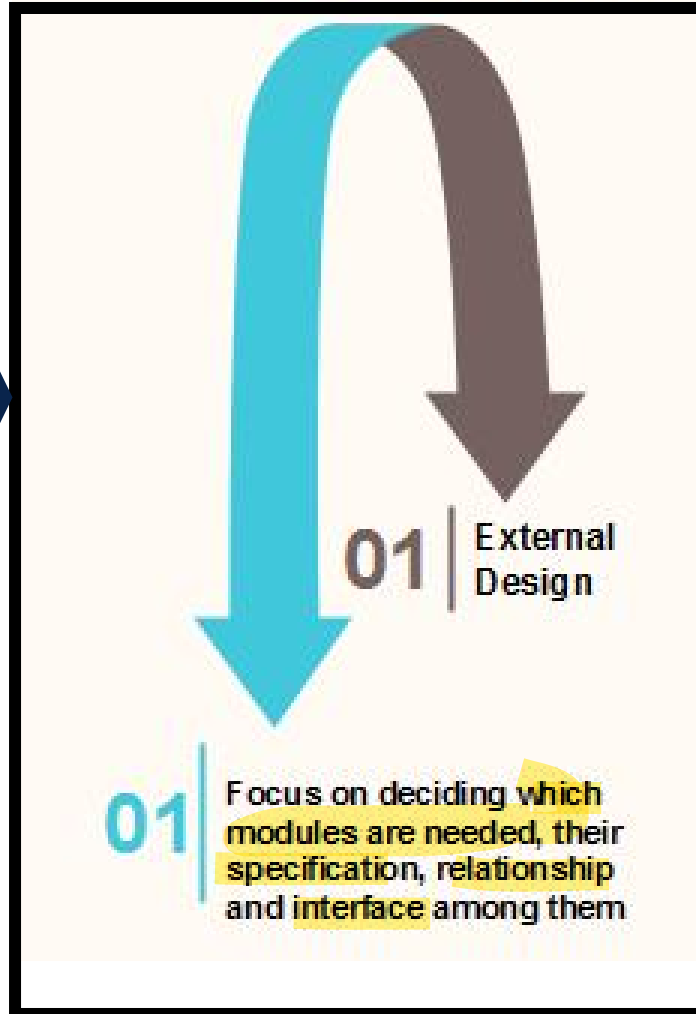
SW DESIGN LEVEL



Software Design Levels

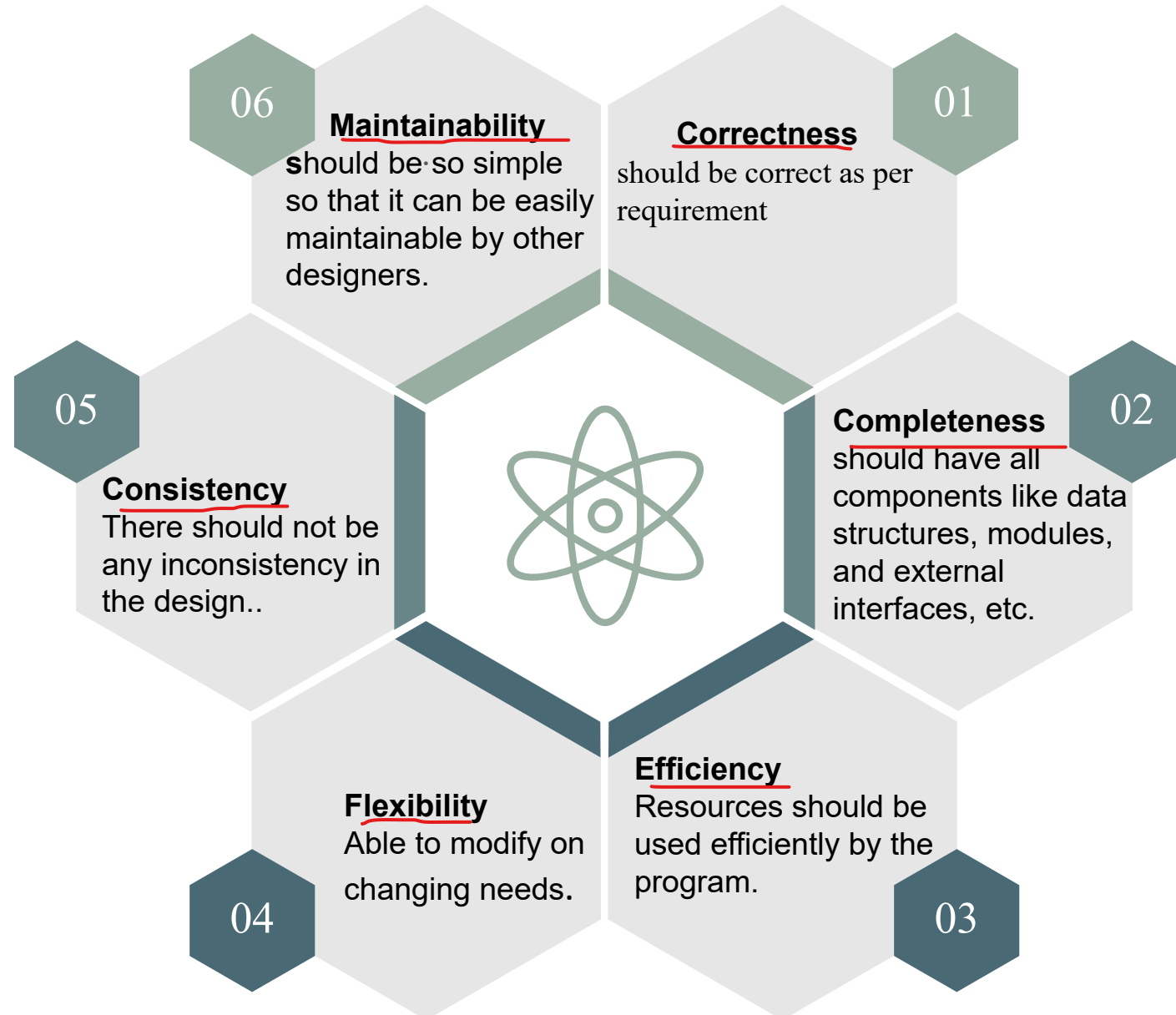
Software design process have two levels:

High Level Design



Detail Level Design

OBJECTIVES OF SW DESIGN



SOFTWARE DESIGN PRINCIPLES



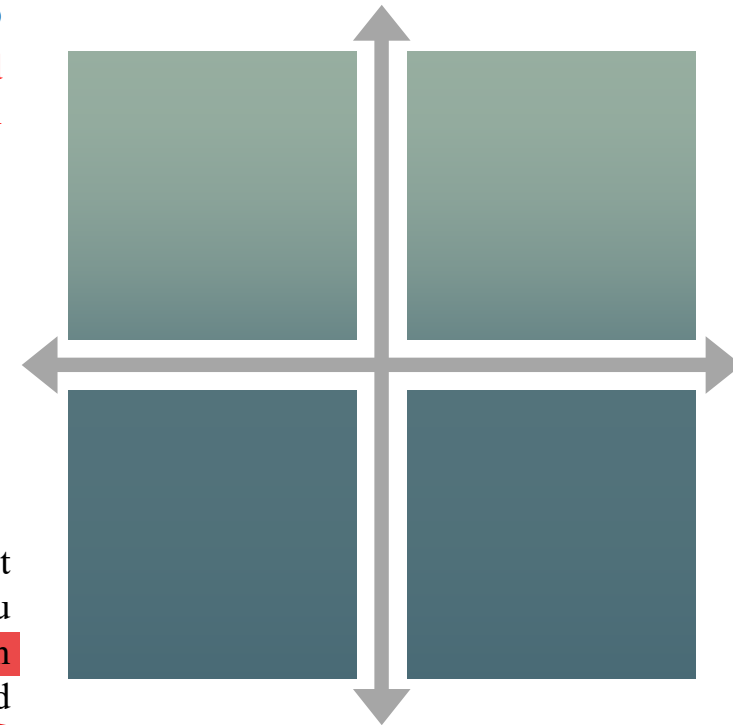
- ✓ Software design principles are concerned with providing means to handle the complexity of the design process effectively.
- ✓ Effectively managing the complexity will not only reduce the effort needed for design but can also reduce the scope of introducing errors during design.

✓ The key software design principles are:

- **SOLID**
Will be discussed in detail

DRY encourages code reusability
KISS emphasizes simplicity in software design
YAGNI advocates for avoiding unnecessary features

- **YAGNI**
You aren't gonna need it
- always implement things when you actually need them
- never implements things before you need them.



DRY

- Don't Repeat Yourself
- each small pieces of knowledge (code) may only occur exactly once in the entire system.
- This helps us to write scalable, maintainable and reusable code.

KISS

- Keep it simple, Stupid!
- keep each small piece of software simple
- unnecessary complexity should be avoided.

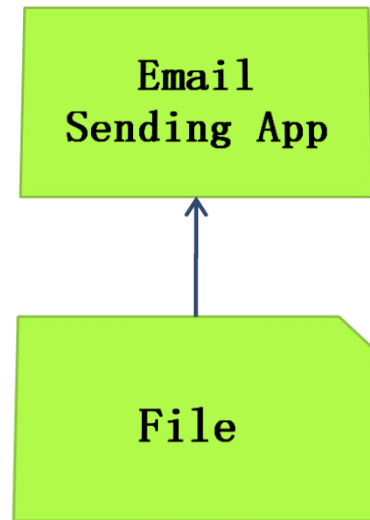
SOFTWARE DESIGN PRINCIPLES - SOLID



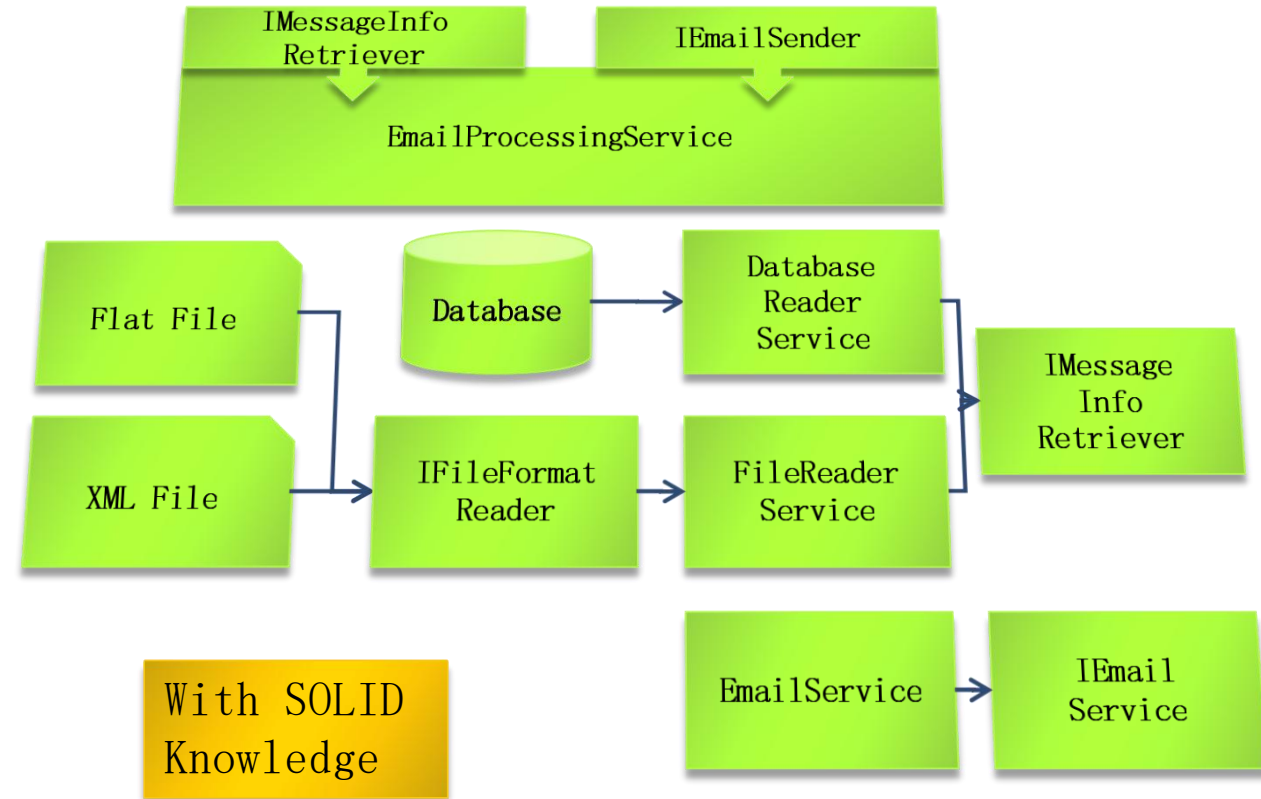
- ✓ In Object Oriented Programming (OOP), SOLID is an acronym, introduced by Michael Feathers, for **five design principles** used to make software design more understandable, flexible, and maintainable.
- ✓ There are five SOLID principles:
 - Single Responsibility Principle (SRP)
 - Open Closed Principle (OCP)
 - Liskov Substitution Principle (LSP)
 - Interface Segregation Principle (ISP)
 - Dependency Inversion Principle (DIP)



WHY SOLID



Without SOLID Knowledge



With SOLID Knowledge

RECAP OO CONCEPT

CLASSES AND OBJECTS



- ✓ A **Class** is a **blueprint or template** from which objects are created
- ✓ A **class** defines **object properties** including a valid range of values, and a default value.
- ✓ A **class** also describes **object behavior**.
- ✓ An **object** is a **member or an "instance" of a class**.
- ✓ Objects are **generated by the classes** and they actually **contain values**.
- ✓ We design an application at the class level. But the code in OOP is organized around

object

Class: Human Object: Man, Woman

Class: Fruit Object: Apple, Banana, Mango, Guava etc.

LET'S TAKE AN EXAMPLE



We want to develop device management system, for example Mobile phone.

how you design such software?

Let's start with few mobiles



Figure out few differences

These also can be listed as their common characteristics

Figure out few common action that can be performed by these devices

LET'S TAKE AN EXAMPLE



COMMON CHARACTERISTICS



- ✓ MANUFACTURER
- ✓ DIMENSION
- ✓ NO. OF CAMERA
- ✓ NO. OF SIM SUPPORTED
- ✓ PRICE

COMMON ACTION

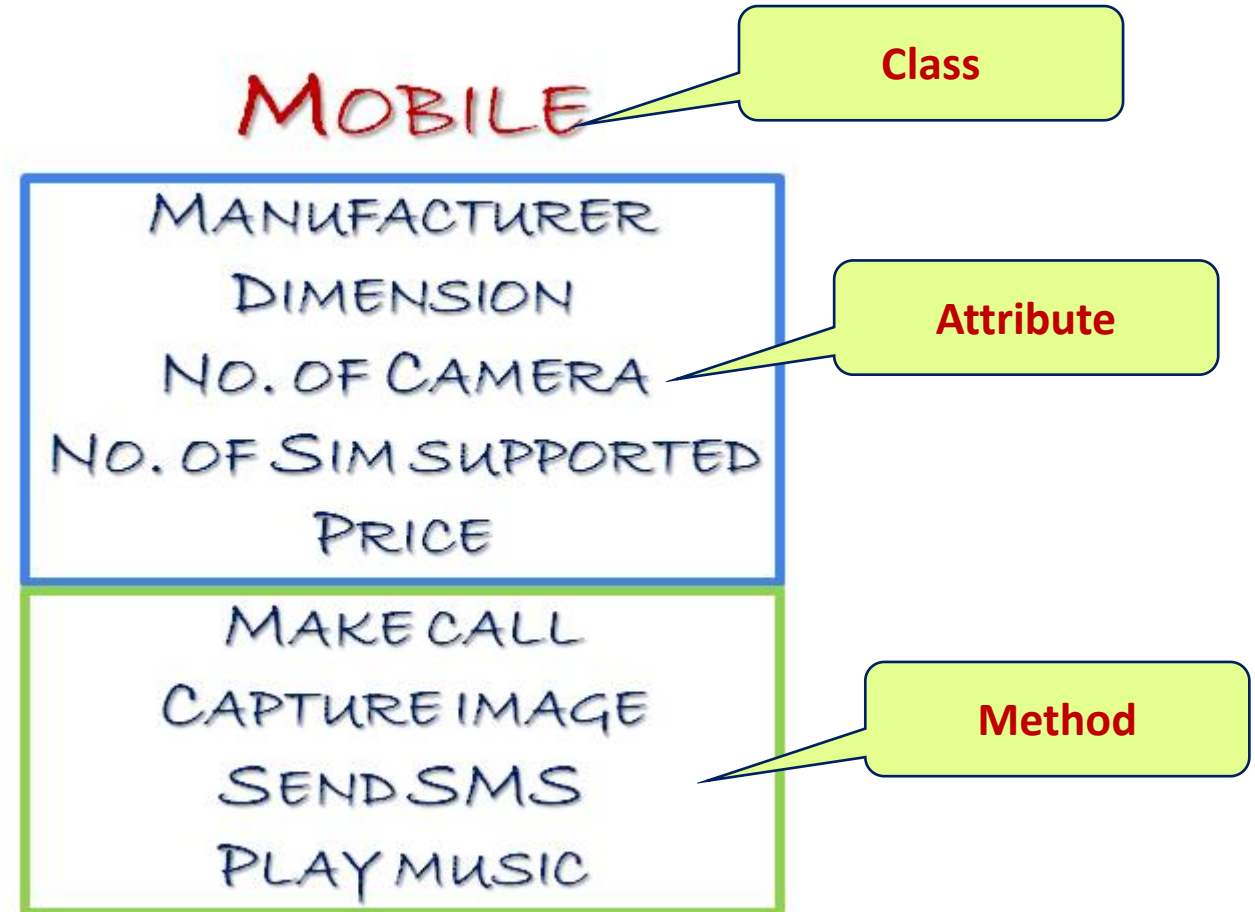


- ✓ MAKE CALL
- ✓ CAPTURE IMAGE
- ✓ SEND SMS
- ✓ PLAY MUSIC

LET'S TAKE AN EXAMPLE



- So far we have defined following things,
- **Class** - Mobile
 - ✓ **Attributes** – Manufacturer, Dimension, No. of Camera, No. of SIM supported, Price
 - ✓ **Methods**- Make call ,Capture image, Send SMS, Play music



LET'S TAKE AN EXAMPLE



Now, for different values of attribute (manufacturer, dimension, no. of camera..etc.) in your class, you will get different mobile objects.



MANUFACTURER = APPLE
DIMENSION = 164.3 X 74.6 X 8.4 MM
NO. OF CAMERA = 3
NO. OF SIM SUPPORTED = 1
PRICE = \$990



MANUFACTURER = SAMSUNG
DIMENSION = 164.3 X 74.6 X 8.4 MM
NO. OF CAMERA = 3
NO. OF SIM SUPPORTED = 2
PRICE = \$440



MANUFACTURER = XIAOMI
DIMENSION = 164.3 X 74.6 X 8.4 MM
NO. OF CAMERA = 3
NO. OF SIM SUPPORTED = 2
PRICE = \$440





ENCAPSULATION

- The ability to protect some components of the object from external entities ("private").
- Encapsulation is achieved when each object keeps its state **private**, inside a class.
- Other objects don't have direct access to this state. Instead, they can only call a list of public functions — called methods.
- Each objects methods manage it's own attributes.
- This is also known as *hiding*.
- An object **A** can learn about the values of attributes of another object **B**, only by invoking the corresponding method (message) associated to the object **B**.



MESSAGE PASSING & ASSOCIATIONS

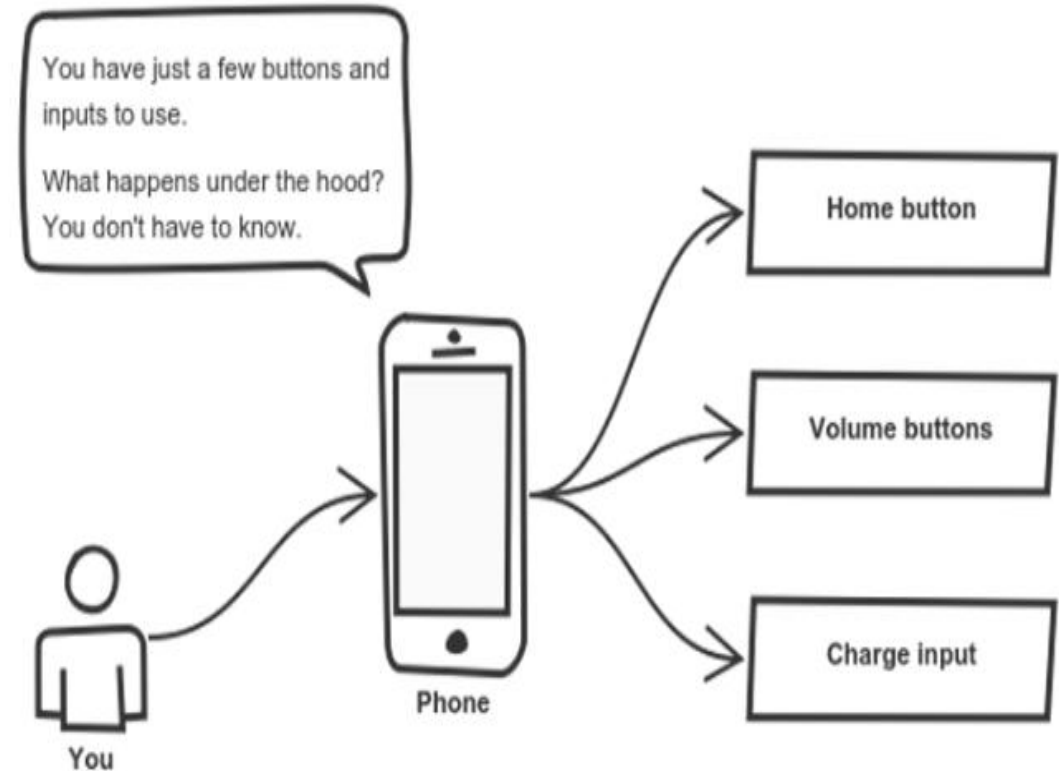
- ✓ Once you have objects, they can interact with each other to make something happen
- ✓ Objects send messages.
- ✓ A **static diagram (class diagram)** shows classes and the **logical associations** between classes, it doesn't show the movement of messages.
- ✓ An **association** between two classes means that the **objects of the two classes can send messages to each other.**

ABSTRACTION



- ✓ Abstraction can be thought of as a natural extension of encapsulation.
- ✓ Its main **goal** is to **handle complexity by hiding unnecessary details** from the user.
- ✓ Applying abstraction means that **each object should only** expose a high-level mechanism for using it.
- ✓ This mechanism should **hide internal implementation details**. It should only reveal operations relevant for the other objects.

Think about your mobile again

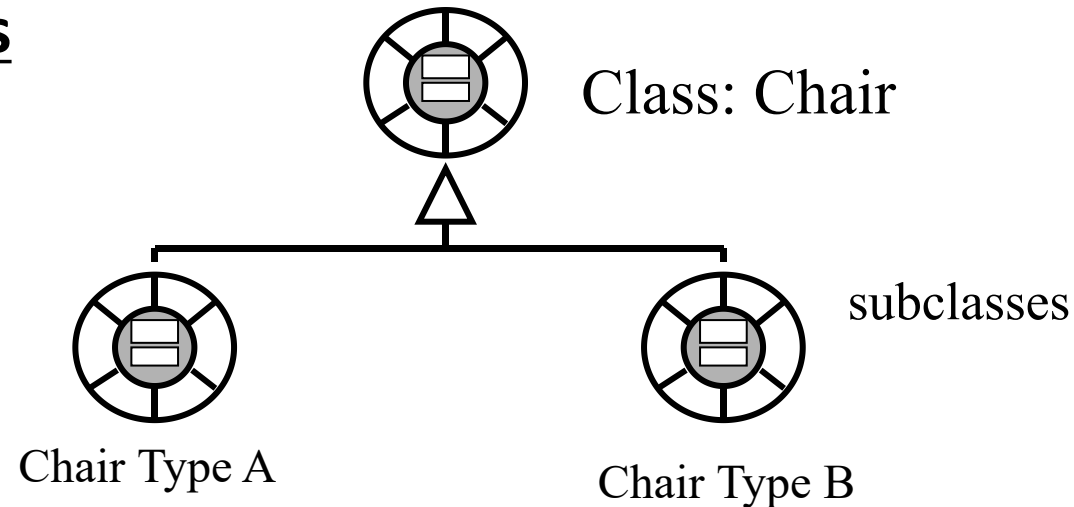




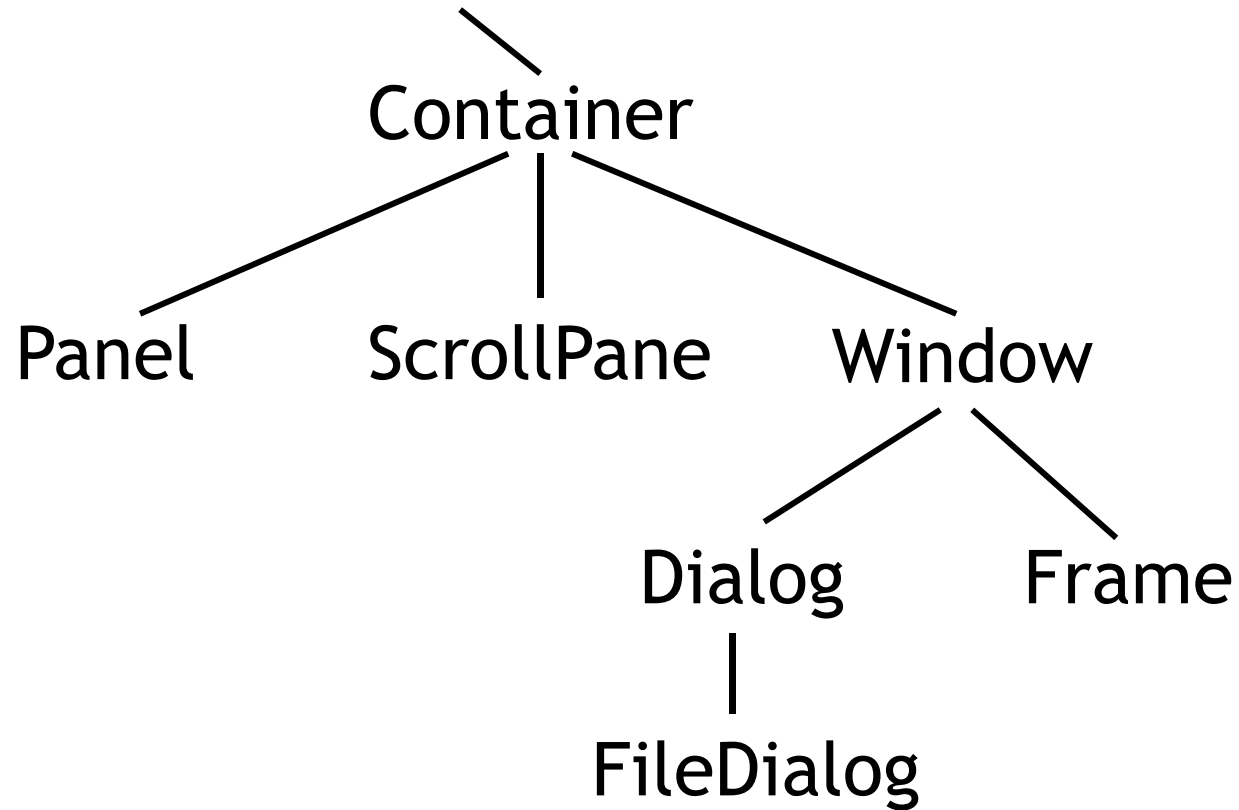
CLASS HIERARCHIES & INHERITANCE

- ✓ Objects are often very similar. They share common logic. But they're not **entirely** the same
- ✓ A class may have several ancestors, up to Object
- ✓ When you define a sub class, you specify its superclass
 - ✓ If you don't specify a superclass, Object is assumed
- ✓ Every class may have one **or more subclasses**
- ✓ Inheritance is important since it leads to the reusability of code.

Class hierarchy diagrams

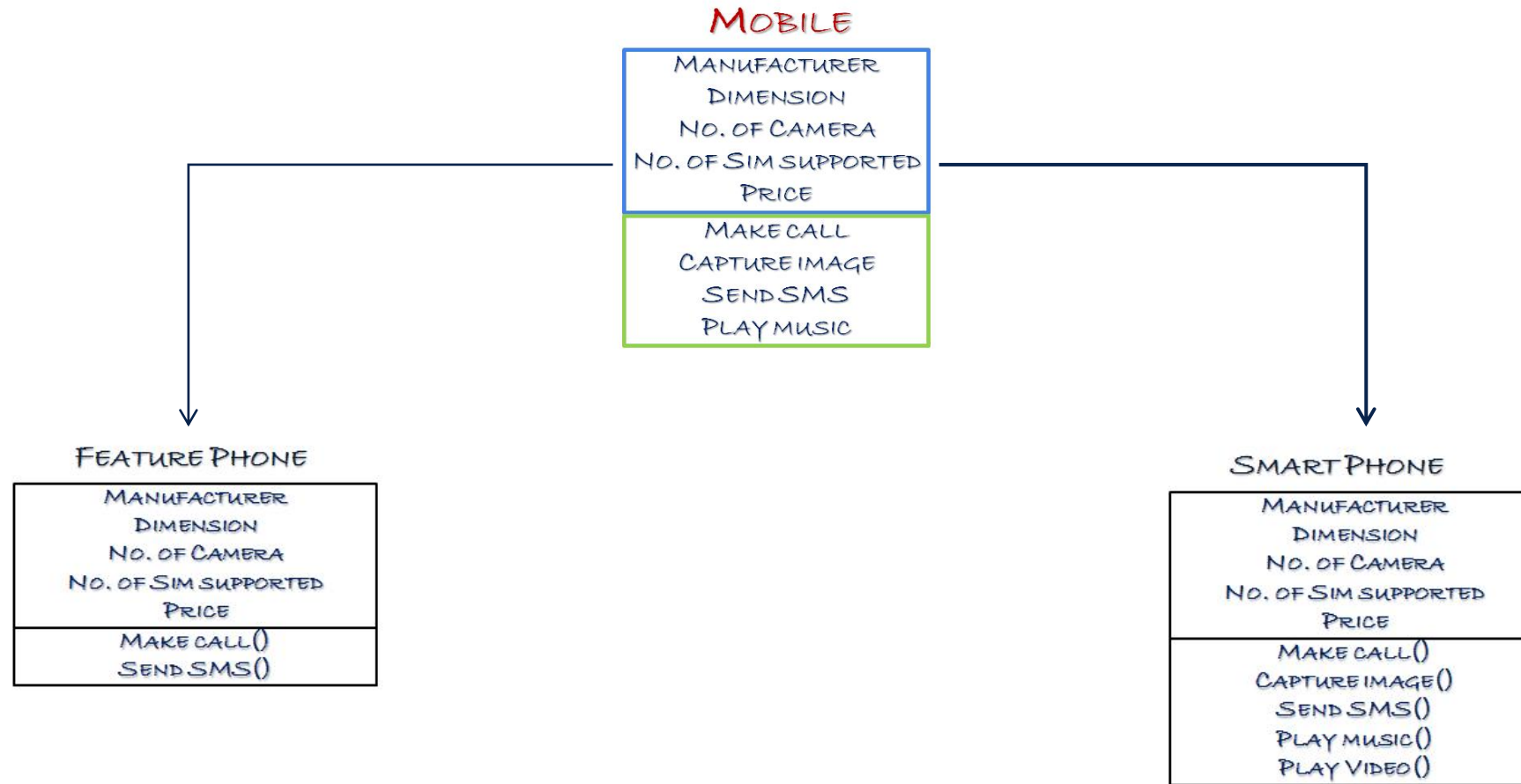


MORE EXAMPLE

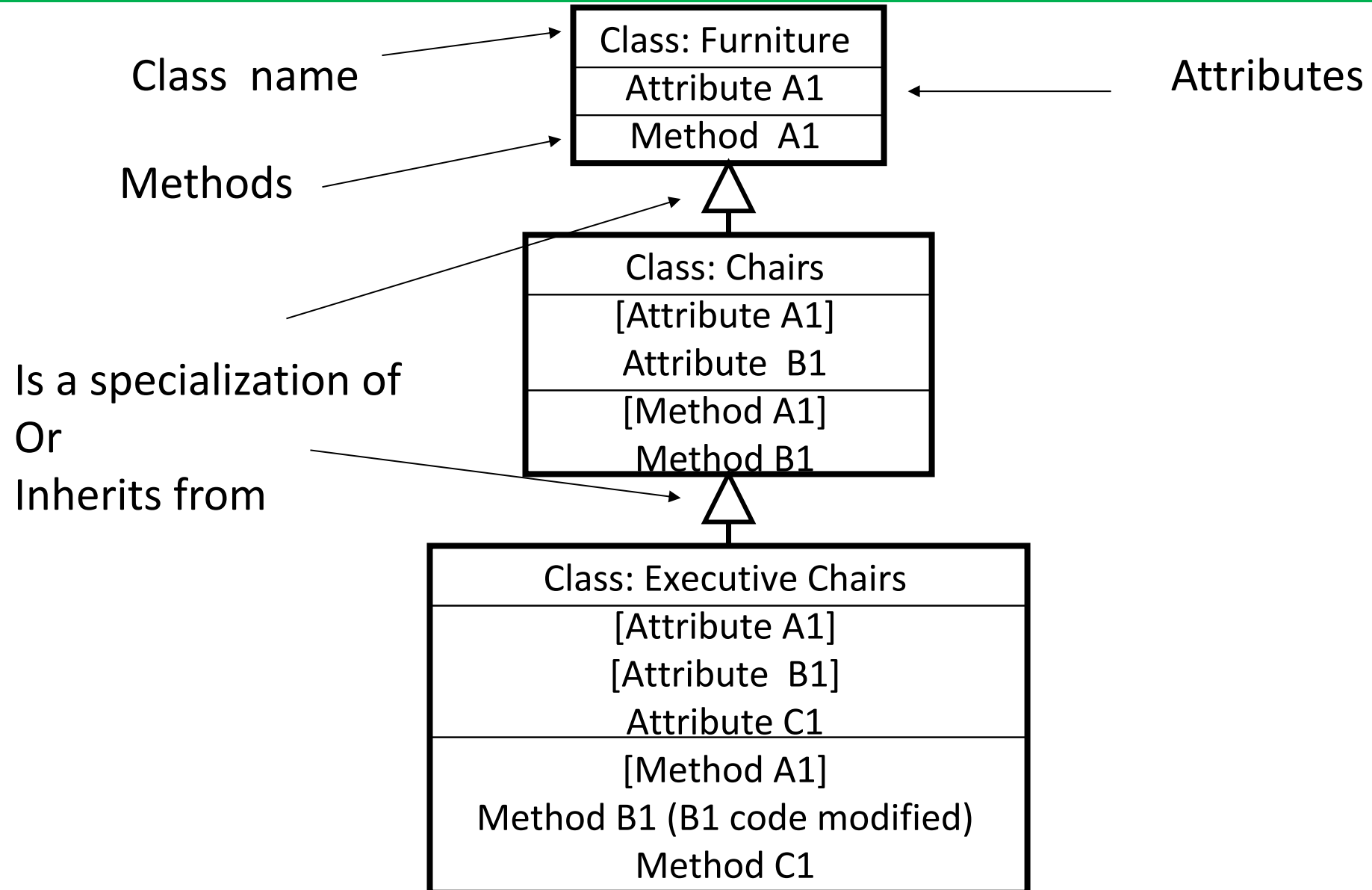


A **FileDialog** is a **Dialog** is a **Window** is a **Container**

RECALL THE MOBILE PHONE EXAMPLE



CLASS INHERITANCE & SPECIALIZATION





PUBLIC, PRIVATE & PROTECTED

✓ Attributes can be public or private:

- Private: it can only be accessed by its own methods
- Public: it can be modified by methods associated with any class

✓ Methods can be public, private or protected:

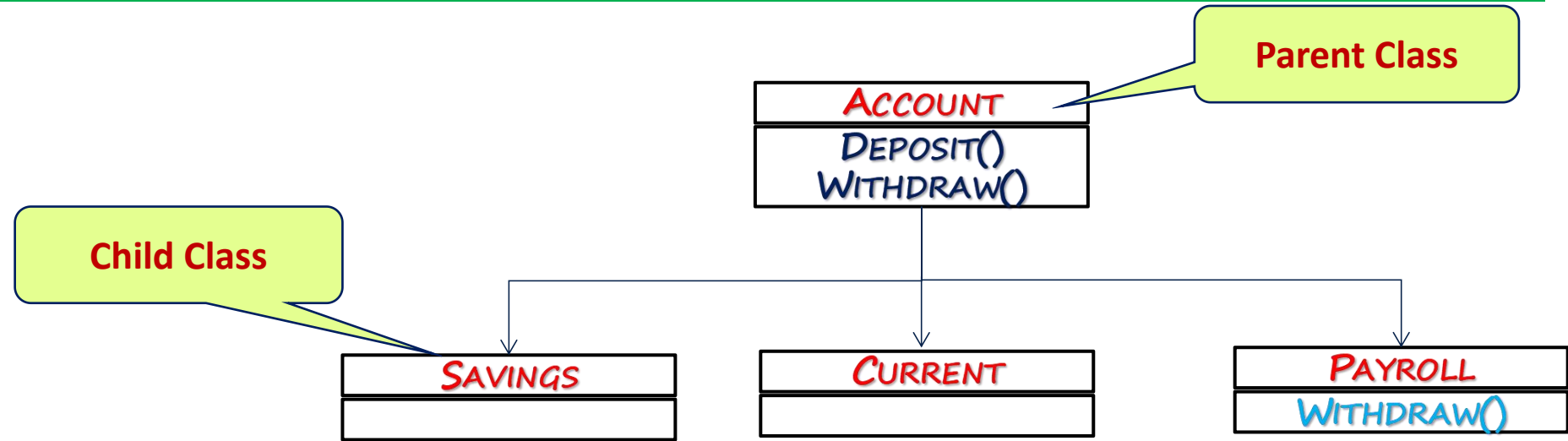
- Public: it's name is exposed to other objects.
- Private: it can't be accessed by other objects, only internally
- Protected: (special case) only subclasses that descend directly from a class that contains it, know and can use this method.

POLYMORPHISM



- ✓ Inheritance lets users inherit attributes and methods, and polymorphism uses these methods to perform different tasks.
- ✓ Means that the same method will behave differently when it is applied to the objects of different classes

LET'S TAKE AN EXAMPLE



The operation of deposit and withdraw is same for Savings and Current accounts. So the inherited methods from Account class will work. However the withdraw method need to be modified for payroll class.

when the "withdraw" method for saving account is called a method from parent account class is executed.

But ,when the "Withdraw" method for the payroll account is called withdraw method defined in the privileged class is executed. This is **Polymorphism in OOPs**.

METHOD OVERRIDING



- Method Overriding is redefining a super class method in a sub class.
- **Rules for Method Overriding**
- The method signature i.e. method name, parameter list and return type have to match exactly.
- The overridden method can widen the accessibility but not narrow it, i.e. if it is private in the base class, the child class can make it public but not vice versa.

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal makes a  
sound");  
    }  
}  
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

METHOD OVERLOADING



- Occurs when a class defines **multiple methods with the same name but different parameters** (number, type, or order of parameters).
- Overloaded methods **enable you to perform different actions based on the arguments passed to them**, providing **flexibility in method usage**.
- Method overloading is about defining multiple methods in a class with the same name but different parameters to perform different actions based on the arguments passed.

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

NEXT CLASS



SOLID AGAIN